

Week10 – Lab09: Serial Communication

PC ____ รหัส ____ ชื่อ-สกุล ____

เปลี่ยนชื่อไฟล์เป็น **“B6501234-นายไมโคร รายงานแลป-Lab09.pdf”**

ส่งงาน Report ที่ >> <https://forms.gle/3HXxoL99mQCi2yky8>

1. MCS51 - Tx: จากโปรแกรม “Lab10Srl1.Asm” ปรับให้วนลูปส่ง “รหัสนักศึกษา, ชื่อ-นามสกุล, ชั้นบรรทัดใหม่-ชิดซ้าย, หน่วงเวลา 500mSec” จำนวน 10 บรรทัด แล้วขึ้นข้อความ “I Want A” ในบรรทัดที่ 11

2. MCS51 - Rx:

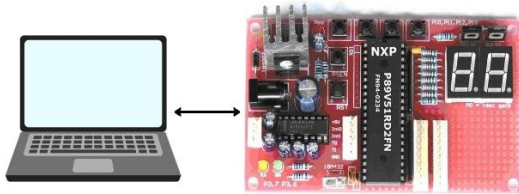
- จากโปรแกรม “Lab10Srl2.Asm” เดิมมีการรับคีย์เฉพาะ A หรือ B ให้ปรับปรุงเพื่อแสดงผลเฉพาะกรณีที่มีการกดคีย์จาก 0-9 และคีย์อื่นๆ ให้แสดง – (ขีดกลาง)
- ถ้ารับค่าคีย์เป็น 1-9 ให้ LED P3.7 กระพริบตามจำนวนที่ป้อนแล้วดับ, ถ้าคีย์อื่นๆ ให้ LED P3.2 ดับ
- ถ้ารับค่าคีย์เป็น 0, +, - หากเป็น {+, 0, -} ให้ 7Segment { นับขึ้น คงค่าการนับ นับลง }

3. MCS51 - Tx, Rx:

- ทดสอบส่งค่าผ่าน Serial-1 ด้วยค่าการกดสวิตช์ SW0 Sw1 Sw2 Sw3 หากกด SW0 ให้ส่ง ASCII ของเลข 0, กด Sw1 = “1”, กด Sw2 = “2” และ กด Sw3 = “3”
- ทดสอบรับค่าจาก Serial-1 หากรับ ASCII เลข 0-9 ให้แสดงเลข 0-9 ที่ 7Segment
- ต่อสาย Tx จากบอร์ด1 ไปยัง Rx ของบอร์ด2 และ GND-GND และทดสอบการทำงาน

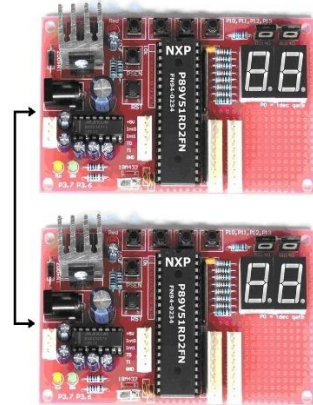
Mission.01 PC ← MCS51

Mission.02 PC → MCS51



Mission.03

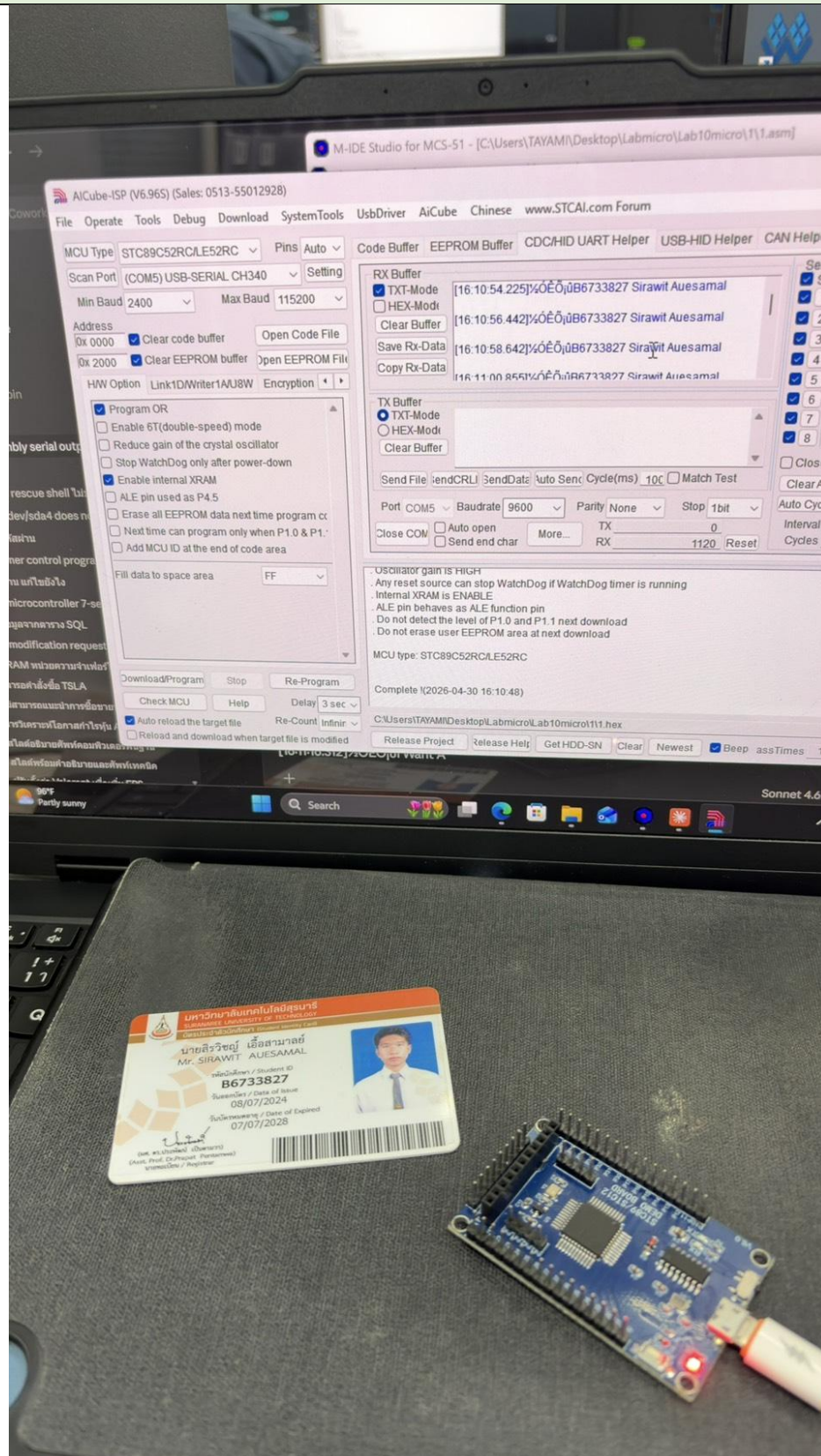
MCS51 → MCS51



1.MCS51 Tx	2.MCS51 Rx	3.MCS51 Tx-Rx	4.ESP32 OTA
5.ESP32 Rx	6.ESP32 Tx + Rx	7. ESP32 -- Modbus (1)	8. ESP32 -- Modbus (2)

1.MCS51 - Tx: จากโปรแกรม “Lab10Srl1.Asm” ปรับให้วนลูปส่ง “รหัสนักศึกษา, ชื่อ-นามสกุล, ชั้นบรรทัดใหม่-ชิดซ้าย, หน่วงเวลา 500mSec” จำนวน 10 บรรทัด แล้วขึ้นข้อความ “I Want A” ในบรรทัดที่ 11

Picture(รูปถ่ายการทำงาน)



MCS51 Capture Code

```

1  ORG      0000H
2          JMP      0100H
3  ORG      0100H
4  MAIN:
5          MOV      SP, #3FH
6          CLR      EA
7          MOV      TMOD, #20H
8          MOV      SCON, #50H
9          MOV      TH1, #0FDH
10         SETB     TR1
11
12         MOV      20H, #10
13  PRINT_INFO:
14         MOV      DPTR, #STR_INFO
15         CALL     SEND_STRING
16         CALL     DELAY_500MS
17         DJNZ     20H, PRINT_INFO
18
19  PRINT_WANT_A:
20         MOV      DPTR, #STR_WANT_A
21         CALL     SEND_STRING
22         CALL     DELAY_500MS
23         SJMP     $
24
25
26  SEND_STRING:
27         CLR      A
28         MOVC     A, @A+DPTR
29         JZ       SEND_STRING_END

```

```

30          CALL    SEND_CHAR
31          INC     DPTR
32          SJMP    SEND_STRING
33 SEND_STRING_END:
34          RET
35
36 SEND_CHAR:
37          CLR     TI
38          MOV     SBUF, A
39          JNB     TI, $
40          CLR     TI
41          RET
42
43 DELAY_500MS:
44          MOV     R5, #20
45 DLY1:     MOV     R6, #200
46 DLY2:     MOV     R7, #250
47          DJNZ    R7, $
48          DJNZ    R6, DLY2
49          DJNZ    R5, DLY1
50          RET
51
52 STR_INFO:
53          DB      'B6733827 Sirawit Auesamal', 0DH, 0AH, 00H
54 STR_WANT_A:
55          DB      'I Want A', 0DH, 0AH, 00H
56          END

```

MCS51 Text Code

```

ORG    0000H
      JMP    0100H
      ORG    0100H
MAIN:
      MOV    SP, #3FH
      CLR    EA
      MOV    TMOD, #20H
      MOV    SCON, #50H
      MOV    TH1, #0FDH
      SETB   TR1

      MOV    20H, #10
PRINT_INFO:
      MOV    DPTR, #STR_INFO
      CALL   SEND_STRING
      CALL   DELAY_500MS

```

```
DJNZ 20H, PRINT_INFO
```

```
PRINT_WANT_A:
```

```
MOV DPTR, #STR_WANT_A
```

```
CALL SEND_STRING
```

```
CALL DELAY_500MS
```

```
SJMP $
```

```
SEND_STRING:
```

```
CLR A
```

```
MOVC A, @A+DPTR
```

```
JZ SEND_STRING_END
```

```
CALL SEND_CHAR
```

```
INC DPTR
```

```
SJMP SEND_STRING
```

```
SEND_STRING_END:
```

```
RET
```

```
SEND_CHAR:
```

```
CLR TI
```

```
MOV SBUF, A
```

```
JNB TI, $
```

```
CLR TI
```

```
RET
```

```
DELAY_500MS:
```

```
MOV R5, #20
```

```
DLY1: MOV R6, #200
```

```
DLY2: MOV R7, #250
```

```
DJNZ R7, $
```

```
DJNZ R6, DLY2
```

```
DJNZ R5, DLY1
```

```
RET
```

```
STR_INFO:
```

```
DB 'B6733827 Sirawit Auesamal', 0DH, 0AH, 00H
```

```
STR_WANT_A:
```

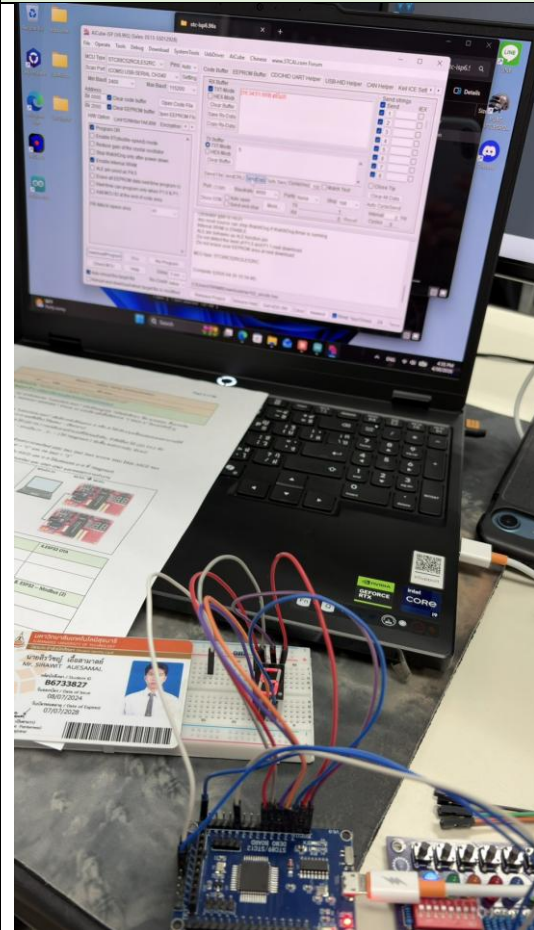
DB 'I Want A', 0DH, 0AH, 00H

END

2. MCS51 - Rx:

- จากโปรแกรม “Lab10Sr12.Asm” เดิมมีการรับคีย์เฉพาะ A หรือ B ให้ปรับปรุงเพื่อแสดงผลเฉพาะกรณีที่มีการกดคีย์จาก 0-9 และคีย์อื่นๆ ให้แสดง – (ขีดกลาง)
- ถ้าวรับค่าคีย์เป็น 1-9 ให้ LED P3.7 กระพริบตามจำนวนที่ป้อนแล้วดับ, ถ้าวรับคีย์อื่นๆ ให้ LED P3.2 ดับ
- ถ้าวรับค่าคีย์เป็น 0, +, - หากเป็น {+, 0, -} ให้ 7Segment { นับขึ้น คงค่าการนับ นับลง }

Picture(รูปถ่ายการทำงาน)



MCS51 Capture Code

```

1  ORG      0000H
2          JMP      0100H
3          ORG      0100H
4  MAIN:
5          MOV      SP, #3FH
6          CLR      EA
7          MOV      TMOD, #20H
8          MOV      SCON, #50H
9          MOV      TH1, #0FDH
10         MOV      TL1, #0FDH
11         SETB     TR1
12         MOV      R0, #0
13         SETB     P3.2
14         SETB     P3.7
15         MOV      A, R0
16         CALL     UPDATE_7SEG
17  WAIT_RX:
18         JNB      RI, WAIT_RX
19         MOV      A, SBUF
20         CLR      RI
21         CJNE     A, #0DH, _N_CR
22         SJMP     WAIT_RX
23  _N_CR:
24         CJNE     A, #0AH, _N_LF
25         SJMP     WAIT_RX
26  _N_LF:
27         CJNE     A, #20H, _N_SP
28         SJMP     WAIT_RX
29  _N_SP:

```



```

30          CJNE    A, #'+', CHK_MINUS
31          INC     R0
32          CJNE    R0, #10, SHOW_CNT
33          MOV     R0, #0
34          SJMP    SHOW_CNT
35  CHK_MINUS:
36          CJNE    A, #'-', CHK_ZERO
37          DEC     R0
38          CJNE    R0, #0FFH, SHOW_CNT
39          MOV     R0, #9
40          SJMP    SHOW_CNT
41  CHK_ZERO:
42          CJNE    A, #'0', CHK_1_TO_9
43          SJMP    SHOW_CNT
44  CHK_1_TO_9:
45          CLR     C
46          MOV     R1, A
47          SUBB    A, #'1'
48          JC      OTHER_KEY
49          MOV     A, R1
50          SUBB    A, #'9'
51          JZ      IS_1_TO_9
52          JNC     OTHER_KEY
53  IS_1_TO_9:
54          SETB    P3.2
55          MOV     A, R1
56          CLR     C
57          SUBB    A, #'0'
58          MOV     R0, A

```

```

60             CALL    UPDATE_7SEG
61 BLINK_LOOP:
62             SETB     P3.7
63             CALL     DELAY_250MS
64             CLR      P3.7
65             CALL     DELAY_250MS
66             DJNZ     R2, BLINK_LOOP
67             SJMP     WAIT_RX
68 OTHER_KEY:
69             CLR      P3.2
70             MOV      A, #10
71             CALL     UPDATE_7SEG
72             SJMP     WAIT_RX
73 SHOW_CNT:
74             CLR      P3.2
75             MOV      A, R0
76             CALL     UPDATE_7SEG
77             SJMP     WAIT_RX
78 UPDATE_7SEG:
79             MOV      DPTR, #SEG_TAB
80             MOVC     A, @A+DPTR
81             MOV      P1, A
82             RET
83 DELAY_250MS:
84             MOV      R5, #5
85 DLY1:        MOV      R6, #100
86 DLY2:        MOV      R7, #228
87             DJNZ     R7, $
88             DJNZ     R6, DLY2

```

89		DJNZ	R5, DLY1
90		RET	
91	SEG_TAB:		
92		DB	3FH ; 0
93		DB	06H ; 1
94		DB	5BH ; 2
95		DB	4FH ; 3
96		DB	66H ; 4
97		DB	6DH ; 5
98		DB	7DH ; 6
99		DB	07H ; 7
100		DB	7FH ; 8
101		DB	6FH ; 9
102		DB	40H ; - cÖ';ÀÒ\$ (segment g à ·èÒ'Ñé¹)
103		END	

MCS51 Text Code

```

ORG 0000H
    JMP 0100H
    ORG 0100H
MAIN:
    MOV SP, #3FH
    CLR EA
    MOV TMOD, #20H
    MOV SCON, #50H
    MOV TH1, #0FDH
    MOV TL1, #0FDH
    SETB TR1
    MOV R0, #0
    SETB P3.2
    SETB P3.7
    MOV A, R0
    CALL UPDATE_7SEG
WAIT_RX:
    JNB RI, WAIT_RX
    MOV A, SBUF
    CLR RI
    CJNE A, #0DH, _N_CR
    SJMP WAIT_RX
_N_CR:
    CJNE A, #0AH, _N_LF

```

```

    SJMP  WAIT_RX
_N_LF:
    CJNE  A, #20H, _N_SP
    SJMP  WAIT_RX
_N_SP:
    CJNE  A, #'+', CHK_MINUS
    INC   R0
    CJNE  R0, #10, SHOW_CNT
    MOV   R0, #0
    SJMP  SHOW_CNT
CHK_MINUS:
    CJNE  A, #'-', CHK_ZERO
    DEC   R0
    CJNE  R0, #0FFH, SHOW_CNT
    MOV   R0, #9
    SJMP  SHOW_CNT
CHK_ZERO:
    CJNE  A, #'0', CHK_1_TO_9
    SJMP  SHOW_CNT
CHK_1_TO_9:
    CLR   C
    MOV   R1, A
    SUBB  A, #'1'
    JC    OTHER_KEY
    MOV   A, R1
    SUBB  A, #'9'
    JZ    IS_1_TO_9
    JNC   OTHER_KEY
IS_1_TO_9:
    SETB  P3.2
    MOV   A, R1
    CLR   C
    SUBB  A, #'0'
    MOV   R0, A
    MOV   R2, A
    CALL  UPDATE_7SEG
BLINK_LOOP:
    SETB  P3.7

```

```
CALL  DELAY_250MS
CLR   P3.7
CALL  DELAY_250MS
DJNZ  R2, BLINK_LOOP
SJMP  WAIT_RX
```

OTHER_KEY:

```
CLR   P3.2
MOV   A, #10
CALL  UPDATE_7SEG
SJMP  WAIT_RX
```

SHOW_CNT:

```
CLR   P3.2
MOV   A, R0
CALL  UPDATE_7SEG
SJMP  WAIT_RX
```

UPDATE_7SEG:

```
MOV   DPTR, #SEG_TAB
MOVC  A, @A+DPTR
MOV   P1, A
RET
```

DELAY_250MS:

```
MOV   R5, #5
```

```
DLY1: MOV   R6, #100
```

```
DLY2: MOV   R7, #228
```

```
DJNZ  R7, $
DJNZ  R6, DLY2
DJNZ  R5, DLY1
RET
```

SEG_TAB:

```
DB    3FH ; 0
DB    06H ; 1
DB    5BH ; 2
DB    4FH ; 3
DB    66H ; 4
DB    6DH ; 5
DB    7DH ; 6
DB    07H ; 7
DB    7FH ; 8
```

```
DB 6FH ; 9
```

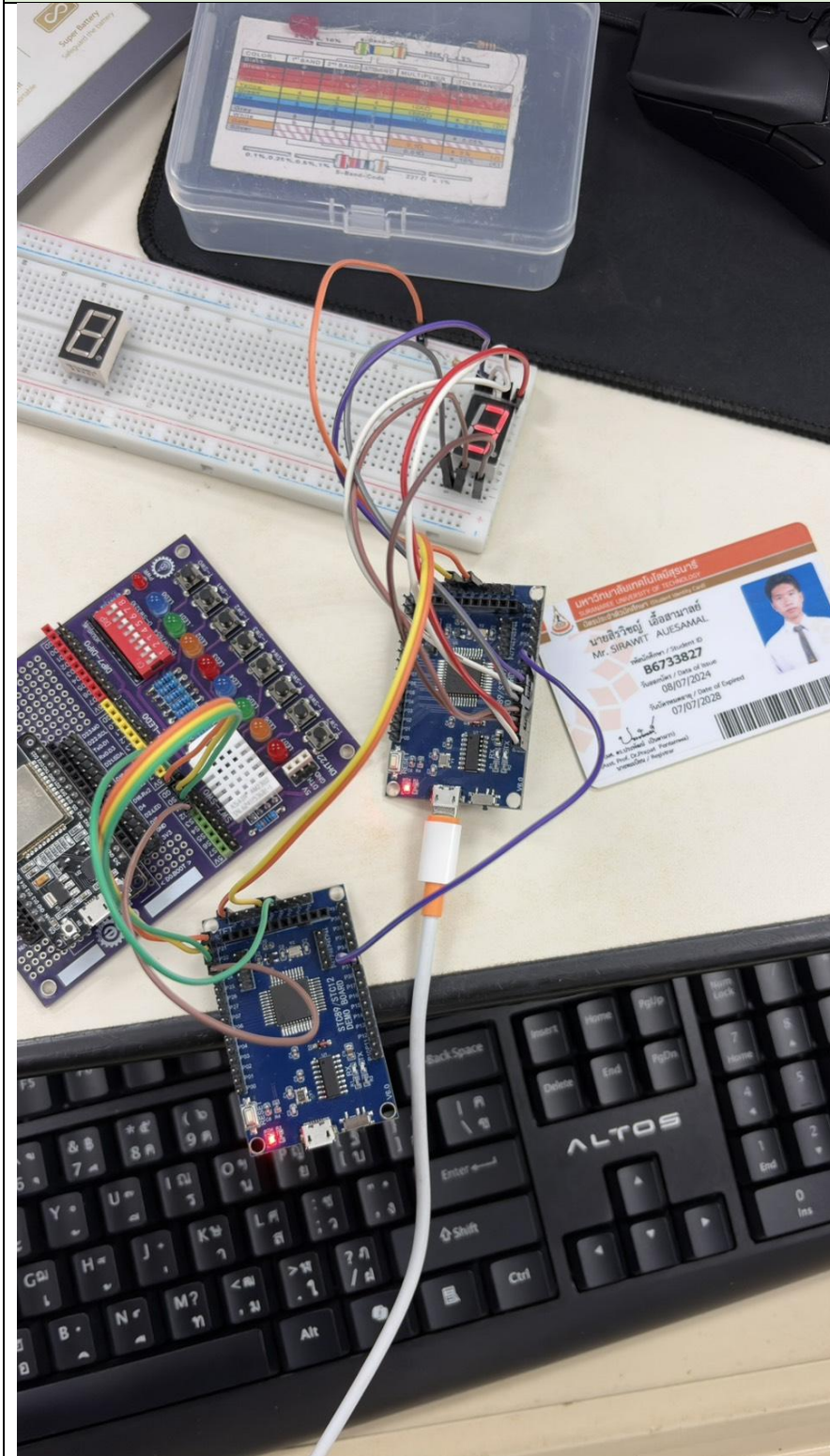
```
DB 40H ; - ขีดกลาง (segment g เท่านั้น)
```

```
END
```

3. MCS51 - Tx, Rx:

- ทดสอบส่งค่าผ่าน Serial-1 ด้วยค่าการกดสวิตช์ SW0 SW1 SW2 SW3 หากกด SW0 ให้ส่ง ASCII ของเลข 0, กด SW1 = "1", กด SW2 = "2" และ กด SW3 = "3"
- ทดสอบรับค่าจาก Serial-1 หากรับ ASCII เลข 0-9 ให้แสดงเลข 0-9 ที่ 7Segment
- ต่อสาย Tx จากบอร์ด1 ไปยัง Rx ของบอร์ด2 และ GND-GND และทดสอบการทำงาน

Picture(รูปถ่ายการทำงาน)



MCS51 Capture Code

```

1  ORG      0000H
2          JMP      0100H
3
4          ORG      0100H
5  MAIN:
6          MOV      SP, #3FH
7          CLR      EA
8          MOV      TMOD, #20H
9          MOV      SCON, #50H
10         MOV      TH1, #0FDH
11         SETB     TR1
12
13         MOV      P1, #00H
14
15  WAIT_RX:
16         JNB      RI, WAIT_RX
17         MOV      A, SBUF
18         CLR      RI
19
20         CLR      C
21         MOV      R1, A
22         SUBB     A, #'0'
23         JC       WAIT_RX
24
25         MOV      A, R1
26         SUBB     A, #'9'
27         JZ       VALID_NUM
28         JNC      WAIT_RX
29
30  VALID_NUM:
31         MOV      A, R1
32         CLR      C
33         SUBB     A, #'0'
34         MOV      DPTR, #SEG_TAB
35         MOVC     A, @A+DPTR
36         MOV      P1, A
37         SJMP     WAIT_RX
38
39  SEG_TAB:
40         DB       3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 6FH
41
42         END

```


1	ORG	0000H
2	JMP	0100H
3		
4	ORG	0100H
5	MAIN:	
6	MOV	SP, #3FH
7	CLR	EA
8	MOV	TMOD, #20H
9	MOV	SCON, #50H
10	MOV	TH1, #0FDH
11	SETB	TR1
12		
13	MOV	P2, #0FFH
14		
15	MAIN_LOOP:	
16	JNB	P2.0, SW0_PRESSED
17	JNB	P2.1, SW1_PRESSED
18	JNB	P2.2, SW2_PRESSED
19	JNB	P2.3, SW3_PRESSED
20	SJMP	MAIN_LOOP
21		
22	SW0_PRESSED:	

```

23             CALL    DELAY_20MS
24             JB      P2.0, MAIN_LOOP
25             MOV     A, #'0'
26             CALL    SEND_TX
27 WAIT_RELO:
28             JNB     P2.0, WAIT_RELO
29             SJMP    MAIN_LOOP
30
31 SW1_PRESSED:
32             CALL    DELAY_20MS
33             JB      P2.1, MAIN_LOOP
34             MOV     A, #'1'
35             CALL    SEND_TX
36 WAIT_REL1:
37             JNB     P2.1, WAIT_REL1
38             SJMP    MAIN_LOOP
39
40 SW2_PRESSED:
41             CALL    DELAY_20MS
42             JB      P2.2, MAIN_LOOP
43             MOV     A, #'2'
44             CALL    SEND_TX

```

45	WAIT_REL2:	
46	JNB	P2.2, WAIT_REL2
47	SJMP	MAIN_LOOP
48		
49	SW3_PRESSED:	
50	CALL	DELAY_20MS
51	JB	P2.3, MAIN_LOOP
52	MOV	A, #'3'
53	CALL	SEND_TX
54	WAIT_REL3:	
55	JNB	P2.3, WAIT_REL3
56	SJMP	MAIN_LOOP
57		
58	SEND_TX:	
59	CLR	TI
60	MOV	SBUF, A
61	JNB	TI, \$
62	CLR	TI
63	RET	
64		
65	DELAY_20MS:	
66	MOV	R6, #40
67	DLY1: MOV	R7, #250
68	DJNZ	R7, \$
69	DJNZ	R6, DLY1
70	RET	
71		
72	END	
MCS51 Text Code		
ORG 0000H		
JMP 0100H		
ORG 0100H		
MAIN:		
MOV SP, #3FH		
CLR EA		
MOV TMOD, #20H		
MOV SCON, #50H		
MOV TH1, #0FDH		

SETB TR1

MOV P1, #00H

WAIT_RX:

JNB RI, WAIT_RX

MOV A, SBUF

CLR RI

CLR C

MOV R1, A

SUBB A, #'0'

JC WAIT_RX

MOV A, R1

SUBB A, #'9'

JZ VALID_NUM

JNC WAIT_RX

VALID_NUM:

MOV A, R1

CLR C

SUBB A, #'0'

MOV DPTR, #SEG_TAB

MOVC A, @A+DPTR

MOV P1, A

SJMP WAIT_RX

SEG_TAB:

DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 6FH

END

ORG 0000H

JMP 0100H

ORG 0100H

MAIN:

```
MOV    SP, #3FH
CLR    EA
MOV    TMOD, #20H
MOV    SCON, #50H
MOV    TH1, #0FDH
SETB   TR1
```

```
MOV    P2, #0FFH
```

MAIN_LOOP:

```
JNB    P2.0, SW0_PRESSED
JNB    P2.1, SW1_PRESSED
JNB    P2.2, SW2_PRESSED
JNB    P2.3, SW3_PRESSED
SJMP   MAIN_LOOP
```

SW0_PRESSED:

```
CALL   DELAY_20MS
JB     P2.0, MAIN_LOOP
MOV    A, #'0'
CALL   SEND_TX
```

WAIT_REL0:

```
JNB    P2.0, WAIT_REL0
SJMP   MAIN_LOOP
```

SW1_PRESSED:

```
CALL   DELAY_20MS
JB     P2.1, MAIN_LOOP
MOV    A, #'1'
CALL   SEND_TX
```

WAIT_REL1:

```
JNB    P2.1, WAIT_REL1
SJMP   MAIN_LOOP
```

SW2_PRESSED:

```
CALL   DELAY_20MS
JB     P2.2, MAIN_LOOP
```

```
MOV  A, #'2'
CALL SEND_TX
WAIT_REL2:
JNB  P2.2, WAIT_REL2
SJMP MAIN_LOOP

SW3_PRESSED:
CALL DELAY_20MS
JB   P2.3, MAIN_LOOP
MOV  A, #'3'
CALL SEND_TX
WAIT_REL3:
JNB  P2.3, WAIT_REL3
SJMP MAIN_LOOP

SEND_TX:
CLR  TI
MOV  SBUF, A
JNB  TI, $
CLR  TI
RET

DELAY_20MS:
MOV  R6, #40
DLY1: MOV  R7, #250
      DJNZ R7, $
      DJNZ R6, DLY1
      RET

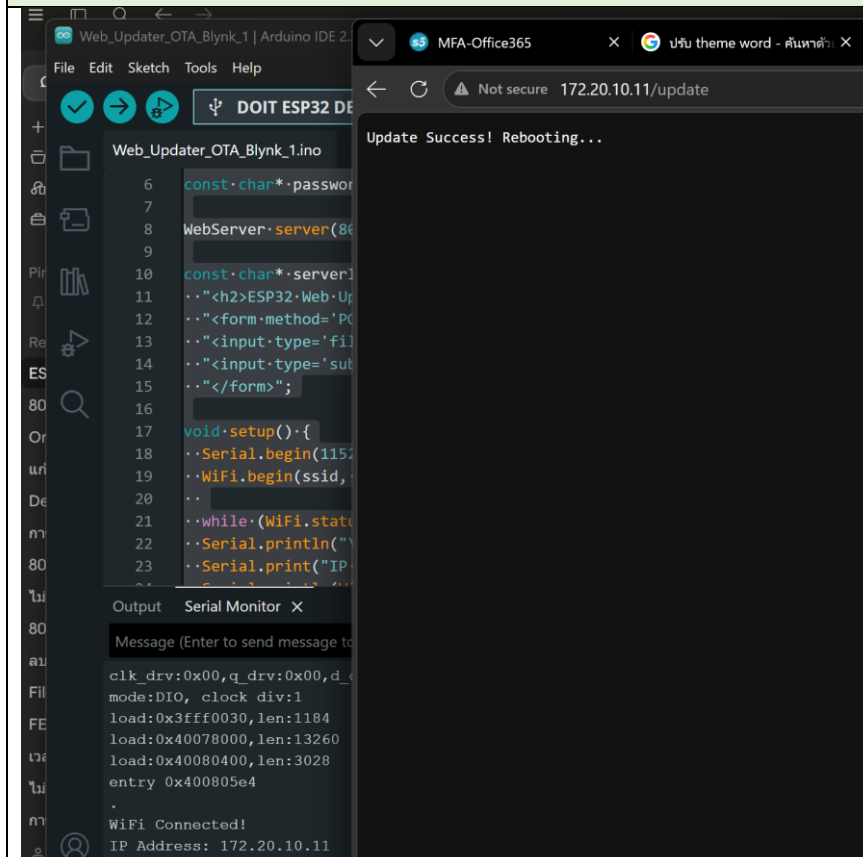
END
```

4 ESP32 - OTA: ทดลอง Upload Code สลับกันไปมาระหว่าง Blink01 กับ Blink02 ด้วยวิธีการทั้ง 4 วิธี

Picture(รูปถ่ายการทำงาน)



Capture หน้าจอขณะ Upload



- 5 ESP32 - Rx: เขียนโปรแกรมสร้างสัญญาณ โดยรับคำสั่งเป็น Serial Command จำนวน 4 คำสั่ง คือ P=xxx<Cr>, D=yy<Cr>, G, ?

Picture(รูปถ่ายการทำงาน)



ESP32 Capture Code

```

5.ino
1  String strData;
2  String Cmd;
3  int Pulse = 50;
4  int Freq = 1;
5
6  unsigned long prevMillis = 0;
7  unsigned long currentMillis;
8
9  float period = 1000;
10 float onTime = 500;
11
12 int ledPin = 2;
13 bool ledState = false;
14
15 void setup() {
16     Serial.begin(115200);
17     pinMode(ledPin, OUTPUT);
18
19     Serial.println("=== READY ===");
20     Serial.println("Use: P=xx, F=xx, G, ?");
21 }
22
23 void loop() {
24     if (Serial.available()) {
25         strData = Serial.readString();
26         strData.trim();
27
28         Cmd = strData.substring(0, 1);
29
30         if (Cmd == "P") {
31             Pulse = strData.substring(2).toInt();
32             Serial.printf("Set Pulse = %d %%\n", Pulse);
33         }
34
35         if (Cmd == "F") {
36             Freq = strData.substring(2).toInt();
37             Serial.printf("Set Freq = %d Hz\n", Freq);
38         }
39
40         if (Cmd == "G") {
41             calculateTime();
42         }
43
44         if (Cmd == "?") {
45             Serial.println("P=xx → Pulse (%)");
46             Serial.println("F=xx → Frequency (Hz)");
47             Serial.println("G → Show result");
48         }
49     }
50
51     currentMillis = millis();
52
53     if (ledState && currentMillis - prevMillis >= onTime) {
54         ledState = false;
55         prevMillis = currentMillis;
56         digitalWrite(ledPin, LOW);
57     }
58     else if (!ledState && currentMillis - prevMillis >= (period - onTime)) {
59         ledState = true;
60         prevMillis = currentMillis;
61         digitalWrite(ledPin, HIGH);
62     }
63 }

```

```

64
65 void calculateTime() {
66     period = 1000.0 / Freq;
67     onTime = (period * Pulse) / 100.0;
68
69     Serial.println("=== CALCULATION ===");
70     Serial.printf("Period = %.2f ms\n", period);
71     Serial.printf("ON time = %.2f ms\n", onTime);
72     Serial.printf("OFF time = %.2f ms\n", period - onTime);
73 }

```

ESP32 Text Code

```

String strData;
String Cmd;

int Pulse = 50;
int Freq = 1;

unsigned long prevMillis = 0;
unsigned long currentMillis;

float period = 1000;
float onTime = 500;

int ledPin = 2;
bool ledState = false;

void setup() {
    Serial.begin(115200);
    pinMode(ledPin, OUTPUT);

    Serial.println("=== READY ===");
    Serial.println("Use: P=xx, F=xx, G, ?");
}

void loop() {
    if (Serial.available()) {
        strData = Serial.readString();
        strData.trim();

        Cmd = strData.substring(0, 1);

        if (Cmd == "P") {

```

```

Pulse = strData.substring(2).toInt();
Serial.printf("Set Pulse = %d %%\n", Pulse);
}

if (Cmd == "F") {
    Freq = strData.substring(2).toInt();
    Serial.printf("Set Freq = %d Hz\n", Freq);
}

if (Cmd == "G") {
    calculateTime();
}

if (Cmd == "?") {
    Serial.println("P=xx → Pulse (%)");
    Serial.println("F=xx → Frequency (Hz)");
    Serial.println("G → Show result");
}
}

currentMillis = millis();

if (ledState && currentMillis - prevMillis >= onTime) {
    ledState = false;
    prevMillis = currentMillis;
    digitalWrite(ledPin, LOW);
}
else if (!ledState && currentMillis - prevMillis >= (period - onTime)) {
    ledState = true;
    prevMillis = currentMillis;
    digitalWrite(ledPin, HIGH);
}
}

void calculateTime() {
    period = 1000.0 / Freq;
    onTime = (period * Pulse) / 100.0;
}

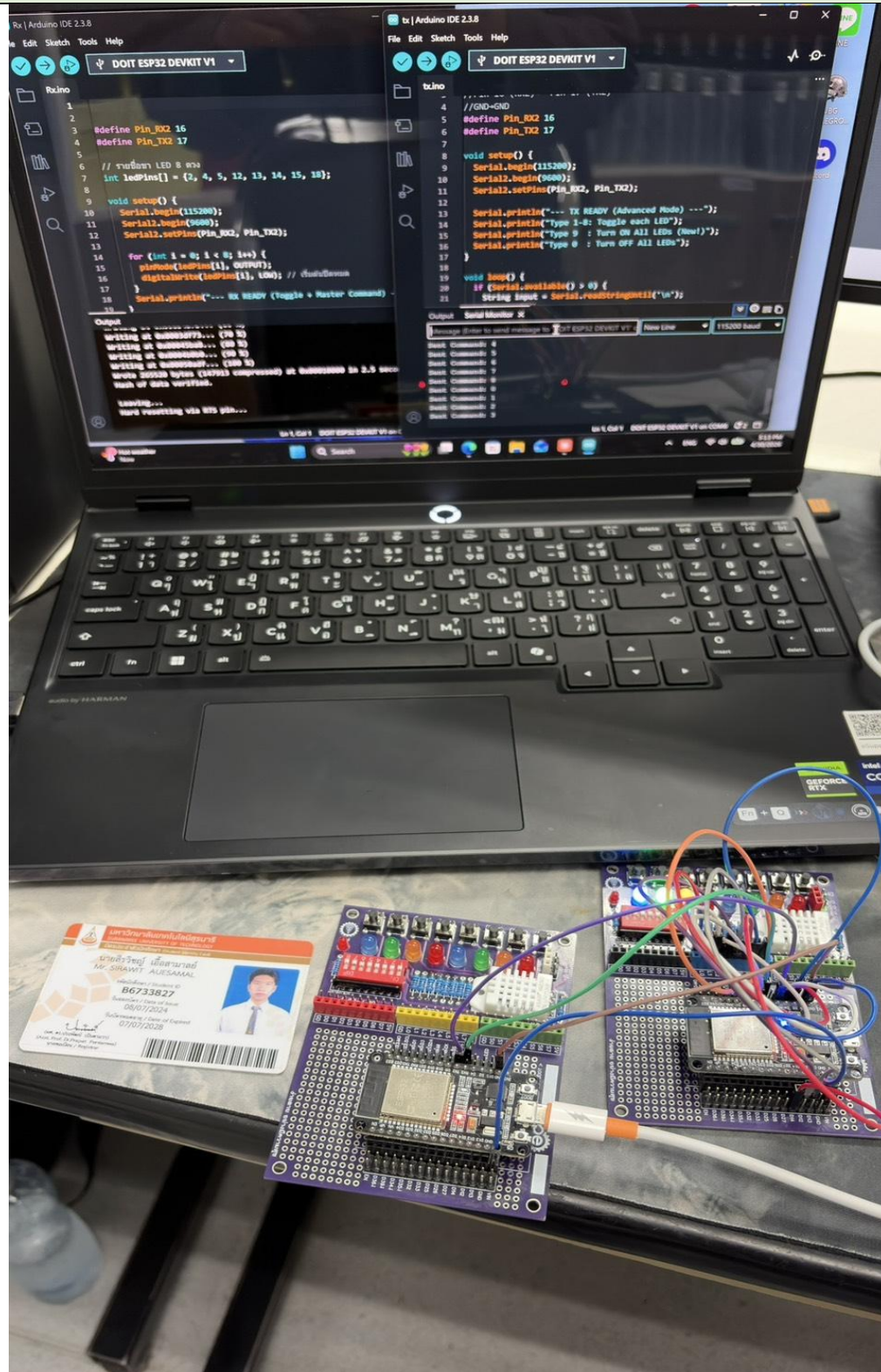
```

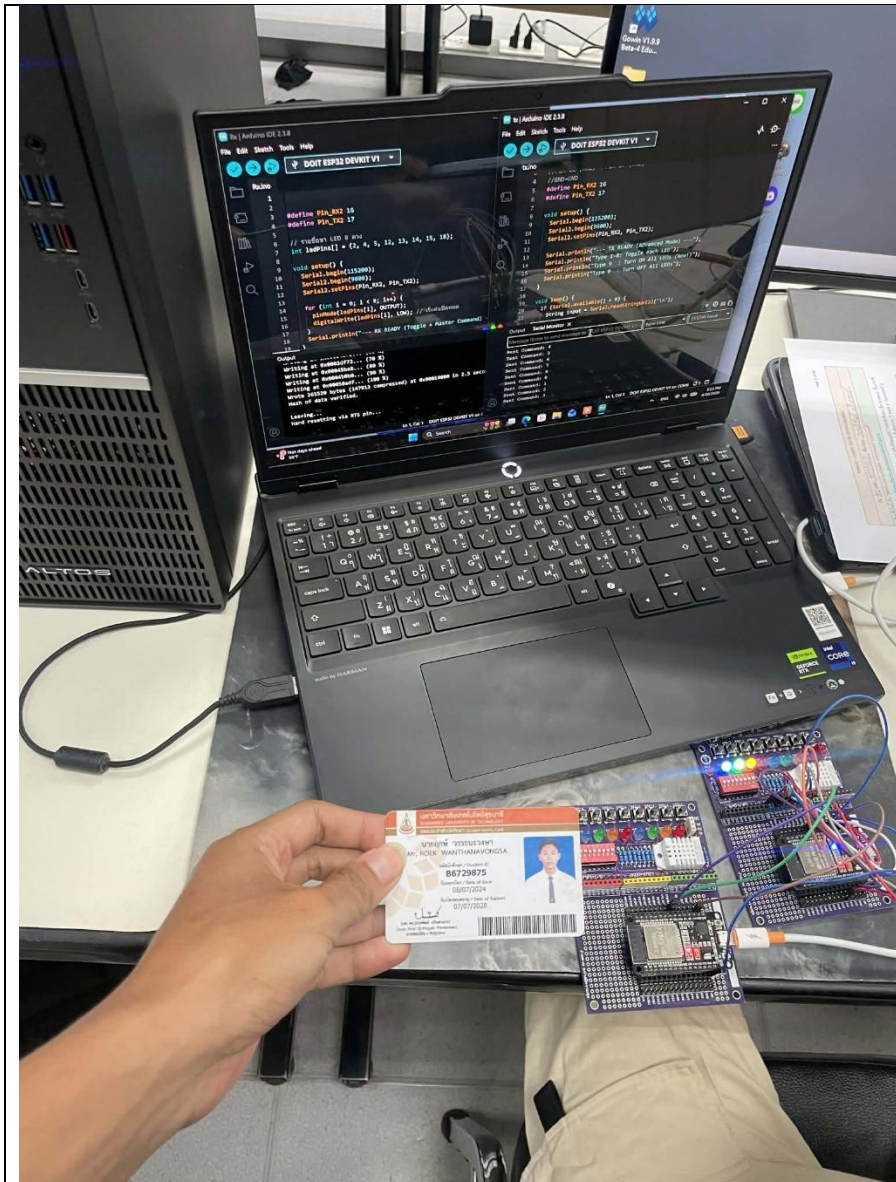
```
Serial.println("=== CALCULATION ===");  
Serial.printf("Period = %.2f ms\n", period);  
Serial.printf("ON time = %.2f ms\n", onTime);  
Serial.printf("OFF time = %.2f ms\n", period - onTime);  
}
```

6 ESP32 - Tx, Rx: ให้นักศึกษากับคู่ระหว่างเครื่องเลขที่ กับ เลขที่+1 ทำการเขียนโปรแกรมร่วมกัน ดังนี้

- PC-คู่ ให้เป็นคนส่ง: ต่ोजจตาม "Digital Input – 8 LED", ใช้ Serial2 ในการรับส่งข้อมูล
- PC-คู่ ให้เป็นคนรับ: ต่ोजจตาม "Digital Output – 8 LED", ใช้ Serial2 ในการรับส่งข้อมูล

Picture(รูปถ่ายการทำงาน)





ESP32-Tx Capture Code

tx.ino

```

1  //บอร์ดส่ง (TX)   →บอร์ดรับ (RX)
2  //Pin 17(TX2) →   Pin 16 (RX2)
3  //Pin 16 (RX2)→   Pin 17 (TX2)
4  //GND→GND
5  #define Pin_RX2 16
6  #define Pin_TX2 17
7
8  void setup() {
9      Serial.begin(115200);
10     Serial2.begin(9600);
11     Serial2.setPins(Pin_RX2, Pin_TX2);
12
13     Serial.println("--- TX READY (Advanced Mode) ---");
14     Serial.println("Type 1-8: Toggle each LED");
15     Serial.println("Type 9   : Turn ON All LEDs (New!)");
16     Serial.println("Type 0   : Turn OFF All LEDs");
17 }
18
19 void loop() {
20     if (Serial.available() > 0) {
21         String input = Serial.readStringUntil('\n');
22         input.trim();
23
24         if (input.length() > 0) {
25             Serial2.println(input);
26             Serial.print("Sent Command: ");
27             Serial.println(input);
28         }
29     }
30 }

```

ESP32-Tx Text Code

```

//บอร์ดส่ง (TX)   →บอร์ดรับ (RX)
//Pin 17(TX2) →   Pin 16 (RX2)
//Pin 16 (RX2)→   Pin 17 (TX2)
//GND→GND
#define Pin_RX2 16
#define Pin_TX2 17

void setup() {
    Serial.begin(115200);
    Serial2.begin(9600);
    Serial2.setPins(Pin_RX2, Pin_TX2);

```



```

Serial.println("--- TX READY (Advanced Mode) ---");
Serial.println("Type 1-8: Toggle each LED");
Serial.println("Type 9 : Turn ON All LEDs (New!)");
Serial.println("Type 0 : Turn OFF All LEDs");
}

void loop() {
  if (Serial.available() > 0) {
    String input = Serial.readStringUntil('\n');
    input.trim();

    if (input.length() > 0) {
      Serial2.println(input);
      Serial.print("Sent Command: ");
      Serial.println(input);
    }
  }
}

```

ESP32-Rx Capture Code

```

Rx.ino
1  #define Pin_RX2 16
2  #define Pin_TX2 17
3
4  // รายชื่อขา LED 8 ดวง
5  int ledPins[] = {2, 4, 5, 12, 13, 14, 15, 18};
6
7  void setup() {
8    Serial.begin(115200);
9    Serial2.begin(9600);
10   Serial2.setPins(Pin_RX2, Pin_TX2);
11
12   for (int i = 0; i < 8; i++) {
13     pinMode(ledPins[i], OUTPUT);
14     digitalWrite(ledPins[i], LOW); // เริ่มต้นปิดหมด
15   }
16   Serial.println("--- RX READY (Toggle + Master Command) ---");
17 }
18
19 void loop() {
20   if (Serial2.available() > 0) {
21     String cmd = Serial2.readStringUntil('\n');
22     cmd.trim();
23
24     int num = cmd.toInt();
25
26     // 1. กรณีพิมพ์ 1-8: สลับสถานะ (Toggle)
27     if (num >= 1 && num <= 8) {
28       int targetPin = ledPins[num - 1];
29       int currentState = digitalRead(targetPin);
30       digitalWrite(targetPin, !currentState);
31
32       Serial.print("LED ");

```



```

33     Serial.print(num);
34     Serial.println(currentState == LOW ? " -> ON" : " -> OFF");
35 }
36 // 2. กดปุ่ม 9: เปิดไฟทุกดวง (Master ON)
37 else if (num == 9) {
38     for (int i = 0; i < 8; i++) {
39         digitalWrite(ledPins[i], HIGH);
40     }
41     Serial.println("Command 9: All LEDs ON");
42 }
43 // 3. กดปุ่ม 0: ปิดไฟทุกดวง (Master OFF)
44 else if (num == 0) {
45     for (int i = 0; i < 8; i++) {
46         digitalWrite(ledPins[i], LOW);
47     }
48     Serial.println("Command 0: All LEDs OFF");
49 }
50 }
51 }

```

ESP32-Rx Text Code

```

#define Pin_RX2 16
#define Pin_TX2 17

// รายชื่อขา LED 8 ดวง
int ledPins[] = {2, 4, 5, 12, 13, 14, 15, 18};

void setup() {
    Serial.begin(115200);
    Serial2.begin(9600);
    Serial2.setPins(Pin_RX2, Pin_TX2);

    for (int i = 0; i < 8; i++) {
        pinMode(ledPins[i], OUTPUT);
        digitalWrite(ledPins[i], LOW); // เริ่มต้นปิดหมด
    }
    Serial.println("--- RX READY (Toggle + Master Command) ---");
}

void loop() {
    if (Serial2.available() > 0) {
        String cmd = Serial2.readStringUntil('\n');
        cmd.trim();

        int num = cmd.toInt();

        // 1. กดปุ่ม 1-8: สลับสถานะ (Toggle)
        if (num >= 1 && num <= 8) {
            int targetPin = ledPins[num - 1];

```

```

int currentState = digitalRead(targetPin);
digitalWrite(targetPin, !currentState);

Serial.print("LED ");
Serial.print(num);
Serial.println(currentState == LOW ? " -> ON" : " -> OFF");
}
// 2. กรณีพิมพ์ 9: เปิดไฟทุกดวง (Master ON)
else if (num == 9) {
    for (int i = 0; i < 8; i++) {
        digitalWrite(ledPins[i], HIGH);
    }
    Serial.println("Command 9: All LEDs ON");
}
// 3. กรณีพิมพ์ 0: ปิดไฟทุกดวง (Master OFF)
else if (num == 0) {
    for (int i = 0; i < 8; i++) {
        digitalWrite(ledPins[i], LOW);
    }
    Serial.println("Command 0: All LEDs OFF");
}
}
}
}

```

7 ESP32 - Modbus Master: Read XY-MD02 (Low Cost RS485 Temperature & Humidity Senso)

Picture(รูปถ่ายการทำงาน)
ESP32-Master Capture Code
ESP32-Master Text Code

8 ESP32 - Modbus Master&Slave: M

Picture(รูปถ่ายการทำงาน)
ESP32-Master Capture Code
ESP32-Master Text Code
ESP32-Slave Capture Code
ESP32-Slave Text Code